FINTECH · COMPLIANCE · PRODUCT SPECIFICATION

# ClearVoice

## E-Invoicing Compliance Gateway for Jordan

Implementation-Ready Product Specification

Prepared for: Obieda Al-Darrabish — Portfolio Engineering Program

# 1. Project Overview

**Project name:** ClearVoice **Tagline:** Compliance, cleared in real time. **Short description:** ClearVoice is a middleware platform that connects any business's billing system to Jordan's national e-invoicing clearance system, automatically generating, signing, submitting, and archiving legally valid invoices.

**Full description:** ClearVoice sits between a business's point-of-sale, ERP, or billing software and the government's e-invoicing clearance service. When a sale happens, ClearVoice receives the invoice data through a simple REST API, converts it into the required UBL 2.1 XML format, digitally signs it, authenticates with the clearance authority using Client ID/Secret credentials, and submits it in real time. The clearance service validates the invoice and returns a signed copy with a QR code — without this round trip, the invoice has no legal standing. ClearVoice stores the cleared invoice, renders a QR-stamped PDF, retries failures automatically, and notifies the client system via webhook the moment an invoice is cleared or rejected. A web dashboard gives finance teams full visibility: every invoice, every clearance attempt, every error, in one auditable timeline.

**Main objective:** Make regulatory e-invoicing compliance a solved problem for any Jordanian business — reduce a multi-week integration project to a single API call.

**Why this product should exist:** Since Regulation No. 2 of 2025 took effect on 1 April 2025, e-invoicing became mandatory in Jordan for all B2B, B2C, and B2G transactions. Jordan uses a Continuous Transaction Control (CTC) model: an invoice is not legally valid until the national system validates it and returns a signed copy with a QR code. Non-compliant invoices are not tax-deductible for the buyer, and penalties can reach 500 JOD per violation. Most small and mid-sized businesses have no engineering team capable of building this integration correctly — schema mapping, digital signatures, retry logic, and audit trails are non-trivial. ClearVoice exists to make this integration a solved, reusable problem instead of something every business reinvents badly.

**What real-world problem it solves:** The gap between "a business needs to issue an invoice" and "that invoice is legally recognized by the state." Today this gap is filled by manual portal entry, fragile custom scripts, or expensive third-party consultants. ClearVoice replaces all three with a dependable, observable, developer-friendly API layer.

**Who suffers from this problem:** Small and mid-sized retailers, clinics, restaurants, consultants, and any VAT-registered business owner who has to issue receipts; the accountants and bookkeepers who face penalties for late or malformed submissions; the software vendors (POS/ERP builders) who now must add compliance features to stay sellable; and buyers who cannot deduct VAT on invoices that were never properly cleared.

**Why current solutions are insufficient:** Manual entry through a government portal does not scale past a handful of invoices a day and introduces human error. Commercial third-party connectors exist but are often expensive, opaque, tightly coupled to specific POS/ERP brands, or lack the developer-friendly API and observability that a technical team actually wants. Very few solutions expose clean documentation, sandbox testing, or a self-serve dashboard.

**Why this project is valuable in Jordan / Amman:** This is not a hypothetical problem — it is a live, mandatory, penalty-backed regulation affecting every registered business in the country as of April 2025. Amman's densest employer cluster is fintech and payments (ProgressSoft, MadfooatCom, HyperPay, MEPS, Dinarak); a project that speaks their exact language — real-time clearance, digital signatures, secure credential handling, retry-safe integrations — is immediately legible to the engineers and hiring managers at those companies.

**Why it is valuable as a software-engineering portfolio project:** It demonstrates production-grade backend engineering (schema mapping, external API integration, retry/idempotency, background workers, webhooks), security discipline (secrets management, signed documents, audit logging), and — critically — that the candidate understands a *real regulatory domain*, not just CRUD. It is the rare junior project that a senior fintech engineer would recognize as "the actual job" rather than a tutorial exercise.

---

# 2. Target Users

### User Type 1: Business Owner / Finance Admin ("Rania, 34, owns a 3-branch pharmacy chain")

- **Who they are:** Non-technical or semi-technical owner/operator responsible for issuing invoices and staying compliant.
- **What they need:** A way to know, at a glance, that every sale today was successfully cleared with the tax authority, and a way to fix anything that wasn't.
- **Pain points:** Fear of penalties; no visibility into whether her POS system's invoices are actually being accepted; dread of month-end reconciliation.
- **What they can do inside the system:** View invoice list and statuses, download cleared PDFs, view/resolve failed invoices, view a compliance summary dashboard, export reports, manage team members.
- **What they cannot do:** Access another tenant's data; modify signed/cleared invoice content (immutability is enforced); bypass validation rules; access raw system logs or developer settings (that's the Admin role).
- **Why they would use the product:** It turns an anxiety-inducing legal obligation into a boring, green-checkmarked dashboard.

### User Type 2: Developer / Integrator ("Yousef, 27, backend developer at a POS software vendor")

- **Who they are:** A technical implementer wiring ClearVoice into an existing billing/POS/ERP system.
- **What they need:** Clear, versioned REST API docs, a sandbox environment, predictable error codes, and webhook events he can build against.
- **Pain points:** Poorly documented government APIs; unclear failure semantics; no test environment to validate before going live.
- **What they can do:** Generate/rotate API keys, configure webhook endpoints, read full OpenAPI documentation, run requests against a sandbox tenant, view request/response logs for debugging.
- **What they cannot do:** Access another tenant's API keys; disable audit logging; skip signature verification in production mode.
- **Why they would use the product:** It is the difference between weeks of guesswork against a government API and an afternoon of integration against clean, documented REST endpoints.

### User Type 3: Tenant Administrator ("Firas, 41, IT lead for a retail group")

- **Who they are:** Manages the ClearVoice account for an organization with multiple branches/users.
- **What they need:** User management, role assignment, branch/tenant configuration, credential rotation, billing/plan visibility.
- **Pain points:** Needs to onboard new staff quickly and offboard departing staff without losing audit history.
- **What they can do:** Invite/remove users, assign roles, configure tenant-level settings (company tax ID, default currency, webhook secret), view full audit log.
- **What they cannot do:** Access ClearVoice's own platform-level admin tools (that is System Operator only).
- **Why they would use the product:** Central control over who can see or touch financial submission data.

### User Type 4: System Operator (internal, ClearVoice platform team)

- **Who they are:** The team operating ClearVoice itself (represented in this portfolio project as a superadmin role).
- **What they need:** Visibility across all tenants for support and reliability — queue health, failure rates, clearance-service uptime.
- **What they can do:** View aggregate system health, inspect (not edit) any tenant's clearance attempts for support purposes, manage feature flags.
- **What they cannot do:** Edit tenant financial data.
- **Why they would use the product:** Operating the platform reliably at scale.

---

# 3. Complete Product Workflow

## 3.1 First-Time User Journey (Business Owner)

1. Rania visits the ClearVoice marketing/landing page and clicks **Start Free Trial**.
2. She enters company name, tax ID, email, and password → clicks **Create Account**.
3. Backend validates the email format and tax-ID format, checks for duplicate tenant, hashes the password (bcrypt/Argon2), creates a `Tenant` and a `User` record with role `TenantAdmin`, and sends a verification email.
4. Rania clicks the verification link → backend marks `User.email_verified = true` and redirects to the onboarding wizard.
5. **Onboarding Step 1 — Company details:** legal name, tax ID, address, default currency (JOD).
6. **Onboarding Step 2 — Integration mode:** "I have a developer / POS vendor" (generates API key immediately) vs. "I want to try the dashboard first" (creates a demo/sandbox tenant seeded with sample invoices).
7. **Onboarding Step 3 — Sandbox walkthrough:** the system auto-submits one sample invoice through the mock clearance service so she sees a cleared invoice with a QR code on her very first screen.
8. She lands on the **Dashboard**, sees "1 invoice cleared" and a green compliance status.

## 3.2 Registration / Login / Password Reset

- Registration: email + password (min 10 chars, at least one number and symbol) + company name + tax ID → email verification required before first login.
- Login: email + password → on success, issue short-lived JWT access token (15 min) + long-lived refresh token (7 days, httpOnly cookie) → on failure, generic "invalid credentials" message (no user enumeration) with exponential lockout after 5 failed attempts.
- Forgot password: user submits email → backend always returns a generic "if that email exists, a reset link was sent" response → time-limited signed reset token emailed → user sets new password → all existing refresh tokens for that user are revoked.

## 3.3 Core Workflow — Submitting and Clearing an Invoice

```
POS/ERP system
  → POST /api/v1/invoices  (invoice payload: buyer, seller, lines, totals)
  → ClearVoice validates payload against schema
  → ClearVoice maps payload to UBL 2.1 XML
  → ClearVoice digitally signs the XML
  → ClearVoice authenticates to clearance service (Client ID + Secret)
  → ClearVoice submits XML to clearance endpoint
      SUCCESS PATH:
        → clearance service returns signed copy + QR code
        → ClearVoice stores ClearanceAttempt(success), Invoice.status = CLEARED
        → ClearVoice generates QR-stamped PDF, stores in object storage
        → ClearVoice fires webhook: invoice.cleared
        → POS/ERP system receives webhook, updates its own record, may print receipt
      FAILURE PATH:
        → clearance service returns validation error (e.g., malformed buyer tax ID)
        → ClearVoice stores ClearanceAttempt(failed, error_code, error_detail)
        → Invoice.status = FAILED
        → ClearVoice enqueues retry job (exponential backoff, max 5 attempts, fresh UUID per attempt)
        → if all retries exhausted: Invoice.status = NEEDS_ATTENTION, webhook invoice.failed fired,
          dashboard shows it in the "Needs Attention" queue
  → Rania (or Yousef via webhook) sees the final state either on the dashboard or in her own system
```

## 3.4 Admin / Tenant Management Workflow

1. Firas logs in → **Settings → Team** → clicks **Invite User** → enters email + role.
2. Backend creates a `User` in `Pending` state, emails an invite link with a signed token.
3. Invitee clicks the link → sets a password → account activates, bound to the inviting tenant.
4. Firas can later change a user's role or deactivate them; deactivation revokes all that user's active sessions immediately.

## 3.5 Background Processes

- **Retry Worker:** polls the `ClearanceAttempt` failed queue every 30 seconds; applies exponential backoff (1m, 5m, 15m, 1h, 6h); marks `NEEDS_ATTENTION` after 5 attempts.
- **Daily Compliance Digest:** scheduled job (cron, 07:00 Amman time) aggregates yesterday's cleared/failed counts per tenant and emails a summary.
- **Webhook Dispatcher:** on any status transition, publishes an event to a queue; a worker delivers it to the tenant's configured webhook URL with HMAC-signed payloads and its own retry policy (separate from clearance retries).
- **Nightly Reconciliation Job:** cross-checks locally stored "CLEARED" invoices against the clearance service's status endpoint (where available) to catch silent drift.

### 3.6 Notifications

- In-app toast + email on: invoice cleared (optional, off by default to avoid noise), invoice needs attention, new team member invited, weekly compliance summary.
- Webhook events: `invoice.submitted`, `invoice.cleared`, `invoice.failed`, `invoice.needs_attention`, `invoice.resubmitted`.

### 3.7 Success / Error States (representative)

- **Success:** green "Cleared" badge, QR-stamped PDF available, timestamp of clearance.
- **Error — validation:** red badge "Rejected — Invalid Buyer Tax ID," inline explanation, "Edit & Resubmit" button.
- **Error — network/timeout to clearance service:** amber badge "Retrying (attempt 2 of 5)," progress shown, no user action needed yet.
- **Error — exhausted retries:** red badge "Needs Attention," manual "Resubmit Now" button available.

---

# 4. Complete Feature List

## MVP

| Feature | What it does | Why it exists | Who uses it | Behavior | On failure | Dependencies |
|---|---|---|---|---|---|---|
| Invoice submission API | Accepts invoice JSON, returns clearance result | Core value proposition | Developers/POS systems | Synchronous validation, async clearance | Returns structured 4xx with field errors | Auth, schema validator |
| UBL 2.1 mapping | Converts internal invoice model to UBL XML | Required format for clearance | System (internal) | Deterministic transform | Logs mapping error, blocks submission | Invoice schema |
| Digital signing | Signs the XML document | Legal requirement for CTC model | System (internal) | Uses stored signing key | Blocks submission, alerts ops | Key management |
| Mock clearance service | Simulates the national clearance API | Enables development without real sandbox access | Developers, demo | Configurable success/fail rates | N/A (it's the test double) | None |
| Retry worker | Re-submits failed invoices automatically | Reliability | System | Exponential backoff, max 5 | Marks NEEDS_ATTENTION | Queue |
| QR-stamped PDF generation | Renders a human-readable invoice with the returned QR | Businesses need paper/PDF proof | Finance users | Generated on clearance success | Falls back to "PDF pending" state | Clearance result |
| Dashboard (invoice list, statuses, filters) | Visibility into all invoices | Trust and compliance oversight | Business owner, admin | Paginated, filterable, searchable | Empty state with CTA | Auth |
| Auth (register/login/JWT/roles) | Secure multi-tenant access | Baseline requirement | All users | JWT + refresh, RBAC | Standard 401/403 | — |
| Audit log | Immutable record of every action | Compliance and trust | Admin | Append-only | N/A | Every write action |

## Version 1.1

| Feature | What it does | Why it exists | Who uses it |
|---|---|---|---|
| Webhooks | Push status changes to client systems | Removes polling burden | Developers |
| API key management UI | Generate/rotate/revoke keys | Security hygiene | Developers, admin |
| Compliance email digest | Daily summary email | Passive reassurance | Business owner |
| Bulk invoice CSV import | Submit many invoices at once | Migration/backfill use case | Finance admin |

## Version 2

| Feature | What it does | Why it exists | Who uses it |
|---|---|---|---|
| Multi-branch reporting | Per-branch breakdown of compliance | Larger retail groups need this | Tenant admin |
| Role-based dashboards | Different views per role | Reduce clutter, focus attention | All |
| Rate-limit tiers / plans | Different quotas per pricing tier | Commercial viability | Platform operator |
| Reconciliation reports (VAT summary) | Aggregate tax figures for filing | Reduces accountant workload | Finance admin/accountant |

## Advanced / Enterprise

| Feature | What it does | Why it exists |
|---|---|---|
| Real clearance-service integration (beyond mock) | Connects to the actual government endpoint | Production readiness |
| Multi-tenant white-labeling | Resell through POS vendors | Business model |
| SSO / SAML login | Enterprise IT requirement | Larger customers |
| Digital certificate management (HSM-backed signing) | Stronger key security | Compliance maturity |

## Optional Future Ideas

- ERP connectors (Odoo, QuickBooks) as prebuilt plugins.
- Arabic-first localized dashboard.
- SMS notifications for critical failures.
- Public status page for clearance-service uptime.

---

# 5. UI / UX Design Specification

**Design style:** Serious, confidence-inspiring, "institutional fintech" — closer to a banking back-office tool than a consumer app. Dense information, generous whitespace around it, no playful illustration, subtle motion only.

**Brand personality:** Trustworthy, precise, calm-under-pressure, understated authority. Think "the audit trail you'd actually trust," not "exciting startup."

**Overall visual direction:** Deep navy as the anchor color evokes banking/regulatory seriousness; a muted gold accent signals "cleared / verified / premium," used sparingly (badges, the QR seal, primary CTA highlights) so it never feels gaudy.

### Color Palette (exact HEX)

| Role | HEX | Usage |
|---|---|---|
| Primary | `#0B2447` | Headers, nav bar, primary buttons, section titles |
| Secondary | `#1B4B91` | Links, secondary buttons, active states |
| Accent | `#C9A24B` | "Cleared" badges, QR seal border, key CTAs, highlights |
| Background | `#F7F8FB` | App background |
| Surface / card background | `#FFFFFF` | Cards, panels, modals |
| Success | `#1E8E5A` | Cleared status, success toasts |
| Warning | `#B8860B` | Retrying / pending status |
| Error | `#C0392B` | Failed / needs-attention status |
| Text — primary | `#12172B` | Body text, headings on light background |
| Text — secondary | `#5B6474` | Captions, helper text, timestamps |
| Text — on-dark | `#F4F6FA` | Text on navy backgrounds |
| Border | `#D8DEE9` | Card borders, table dividers |
| Row alt (tables) | `#F4F6FA` | Zebra striping |

### Typography

- **Recommended fonts:** Heading — *Source Serif Pro* or *IBM Plex Serif* (institutional gravitas). Body/UI — *Inter* or *IBM Plex Sans* (clean, highly legible at small sizes for dense tables). Monospace (for API keys, invoice IDs) — *IBM Plex Mono*.
- **Heading sizes:** H1 28px/2.0rem (page titles), H2 20px/1.4rem (section headers), H3 16px/1.15rem (card titles).
- **Body sizes:** Base 14px, small/caption 12px, table text 13px.
- **Font weights:** Headings 600–700; body 400; emphasis/labels 500.
- **Line spacing:** 1.5 for body text, 1.3 for headings, 1.4 for table rows.

### Spacing System

- **Base unit:** 4px grid.
- **Page margins:** 32px desktop, 16px mobile.
- **Card padding:** 20px.
- **Card-to-card gap:** 16px.
- **Component spacing (form fields):** 16px vertical gap.
- **Border radius:** 8px for cards/inputs, 6px for buttons, 999px (pill) for status badges.
- **Button sizes:** Default height 40px, compact 32px, large (primary CTA) 48px.

### Components

- **Buttons:** Primary — solid navy `#0B2447`, white text, 6px radius, subtle shadow on hover (darken 8%). Secondary — outlined navy, transparent fill. Destructive — solid `#C0392B`. Gold accent button reserved for the single highest-priority action per screen (e.g., "Resubmit Now").

- **Inputs:** White background, 1px `#D8DEE9` border, 8px radius, focus state = 2px `#1B4B91` outline + subtle glow. Error state = red border + inline message below.
- **Dropdowns:** Same shell as inputs, chevron icon right-aligned, options list with 8px radius, hover row `#F4F6FA`.
- **Tables:** Header row navy background/white text (sticky on scroll), zebra striping, row hover `#EDF1F8`, status shown as pill badges (green/gold/red), right-aligned numeric columns.
- **Cards:** White surface, 8px radius, 1px `#D8DEE9` border, subtle shadow (0 1px 3px rgba(0,0,0,0.06)).
- **Modals:** Centered, max-width 560px, dimmed navy overlay at 60% opacity, 12px radius, header/body/footer sections, footer right-aligned action buttons.
- **Toast notifications:** Top-right, 4-second auto-dismiss (persistent for errors), colored left border (4px) matching status, white background.
- **Tabs:** Underline style, active tab navy underline + navy text, inactive gray text, transition 150ms.
- **Navigation (top bar):** Fixed, navy background, white logo/wordmark, right-aligned user menu + tenant switcher.
- **Sidebar:** 240px wide, white background, navy active-item highlight (left 3px accent bar + light navy background tint), collapsible to icon-only 64px on tablet.
- **Search:** Persistent top-bar search for invoice ID/buyer name, instant results dropdown.
- **Filters:** Chip-style filter bar above tables (status, date range, branch), active filters shown as removable pills.
- **Pagination:** Numbered pagination with prev/next, 25/50/100 per-page selector, "showing X–Y of Z" label.
- **Charts:** Line chart for cleared-vs-failed trend (navy + red lines), donut chart for status breakdown (green/gold/red), all using the palette above, no gradients, minimal gridlines.

---

# 6. Complete Screen List

## 6.1 Landing Page

- **Purpose:** Convert visiting SMEs/developers into signups.
- **Layout:** Full-width hero with navy gradient background, headline + subhead + "Start Free" CTA (gold accent), below-fold: 3-step "how it works" strip, logos/trust bar, pricing teaser, footer.
- **Empty/loading/error states:** N/A (static marketing content).
- **Mobile:** Stacks vertically, hero CTA remains above the fold.

## 6.2 Login

- **Purpose:** Authenticate returning users.
- **Layout:** Centered card (400px) on light background, logo top, email/password fields, "Forgot password?" link, primary login button, "Don't have an account? Sign up" footer link.
- **Error state:** Inline red text under the form: "Invalid email or password."
- **Loading state:** Button shows spinner, disabled during request.
- **Mobile:** Card becomes full-width with 16px side padding.

## 6.3 Register

- **Purpose:** Create a new tenant + admin user.
- **Layout:** Centered card, fields: company name, tax ID, email, password, confirm password, terms checkbox, submit button.
- **Validation states:** Real-time inline validation (password strength meter, tax-ID format check).

## 6.4 Forgot / Reset Password

- **Purpose:** Self-service credential recovery.
- **Layout:** Single-field form → confirmation screen ("check your email") → (via link) new-password form.

## 6.5 Onboarding Wizard

- **Purpose:** Guide first-time setup.
- **Layout:** 3-step horizontal progress indicator (navy filled circles), one step per screen, "Back/Next" navigation, final step auto-redirects to dashboard with a celebratory but restrained success state (checkmark, not confetti — matches brand seriousness).

## 6.6 Dashboard (Home)

- **Purpose:** At-a-glance compliance status.
- **Layout:** Top row of 4 stat cards (Total Invoices, Cleared, Needs Attention, Clearance Rate %), below: trend line chart (30-day cleared vs failed), below: "Needs Attention" table (top 5, with "View All" link), sidebar navigation persistent left.
- **Empty state:** Illustration-free, text: "No invoices yet — submit your first invoice via the API or try the sandbox." with a "View API Docs" button.
- **Loading state:** Skeleton cards (gray pulsing blocks) matching final layout shape.

## 6.7 Invoices List

- **Purpose:** Full searchable/filterable invoice history.
- **Layout:** Filter bar top (status, date range, branch, search), data table (Invoice #, Buyer, Amount, Status badge, Cleared At, Actions), pagination footer.
- **Row actions:** View, Download PDF, Resubmit (if failed).
- **Empty/error states:** "No results for these filters" with "Clear filters" link; on API error, a retry banner.

## 6.8 Invoice Detail

- **Purpose:** Full record of one invoice's lifecycle.
- **Layout:** Header (invoice #, status badge, buyer/seller summary), tabbed body: **Overview** (line items table, totals), **Clearance History** (timeline of every attempt with timestamps and error detail), **Audit Log** (who/what/when), **Document** (embedded PDF preview + QR).
- **Actions:** Resubmit, Download PDF, Copy Invoice UUID.

### 6.9 Needs Attention Queue

- **Purpose:** Focused worklist for failed invoices.
- **Layout:** Same table pattern as Invoices List, pre-filtered, with a prominent "Resubmit All" bulk action and per-row error-reason column.

### 6.10 API Keys / Developer Settings

- **Purpose:** Manage programmatic access.
- **Layout:** List of keys (name, prefix shown, created date, last used, revoke button), "Generate New Key" modal (key shown once, copy-to-clipboard, warning it won't be shown again).

### 6.11 Webhooks Settings

- **Purpose:** Configure event delivery endpoints.
- **Layout:** Endpoint URL field, event-type checkboxes, signing-secret display, "Send Test Event" button, delivery log table below (status code, timestamp, retry count).

### 6.12 Team / Users

- **Purpose:** Manage tenant members and roles.
- **Layout:** Table of users (name, email, role dropdown, status, remove action), "Invite User" button opening a modal.

### 6.13 Settings — Company Profile

- **Purpose:** Tenant configuration.
- **Layout:** Form sections: Company Details, Tax Configuration, Notification Preferences, Danger Zone (deactivate account).

### 6.14 Analytics / Compliance Reports

- **Purpose:** Aggregate reporting for accountants/management.
- **Layout:** Date-range picker, summary cards (total cleared value, VAT total), exportable table, downloadable CSV/PDF report.

### 6.15 Admin — Platform Overview (System Operator only)

- **Purpose:** Cross-tenant operational health (internal).
- **Layout:** Aggregate charts (total volume, error rate by tenant, queue depth), incident banner if clearance service is degraded.

### 6.16 Error Pages (404 / 500 / 403)

- **Purpose:** Graceful failure.
- **Layout:** Centered, navy icon mark, short message, "Back to Dashboard" button.

---

# 7. Responsive Design

- **Desktop (≥1280px):** Full sidebar (240px) + content area; tables show all columns; charts full-width side-by-side where paired.
- **Laptop (1024–1279px):** Sidebar remains, tables drop lowest-priority column (e.g., "Branch") behind a "more" expandable row.
- **Tablet (768–1023px):** Sidebar collapses to icon-only (64px) with tooltip labels on hover/tap; stat cards go from 4-across to 2x2 grid; tables become horizontally scrollable within their card.
- **Mobile (≤767px):** Sidebar becomes a bottom nav bar (Dashboard, Invoices, Attention, Settings) or slide-out drawer via hamburger; tables convert to stacked "card list" rows (each invoice becomes a card with label:value pairs instead of columns); forms go single-column full-width; modals become full-screen sheets; filter bar collapses into a single "Filters" button opening a bottom sheet; charts simplify to single-series view with a toggle instead of side-by-side comparison; touch targets minimum 44x44px; swipe-to-reveal actions (Resubmit) on invoice cards.

# 8. Frontend Architecture

- **Framework:** React 18 with TypeScript.
- **Language:** TypeScript throughout (strict mode).
- **UI component system:** A lightweight design-system layer built on **shadcn/ui** (Radix primitives + Tailwind CSS) themed with the ClearVoice palette above — gives accessible primitives (dialogs, dropdowns, tabs) without heavy bundle weight.
- **State management: TanStack Query (React Query)** for all server state (invoices, tenants, users) — handles caching, retries, and background refetch natively, which fits an app that's fundamentally "read a lot of status data." **Zustand** for small pieces of client-only UI state (sidebar collapsed, active filters).
- **API layer:** A typed API client generated from the backend's OpenAPI spec (using `openapi-typescript` + a thin fetch wrapper), so frontend and backend never drift out of sync on request/response shapes.
- **Authentication handling:** Access token held in memory (not localStorage, to reduce XSS exposure); refresh token in an httpOnly, Secure, SameSite=Strict cookie; an Axios/fetch interceptor auto-refreshes on 401 and retries the original request once.
- **Form handling: React Hook Form** for performant, uncontrolled forms.
- **Validation: Zod** schemas shared conceptually with backend validation rules (mirrored, not literally shared across languages, but kept in lockstep) — gives typed, composable client-side validation with clear error messages.
- **Routing: React Router v6**, nested layouts (AuthLayout, DashboardLayout with persistent sidebar/topbar), route-level code splitting via `React.lazy`.

### Suggested Frontend Folder Structure

```
frontend/
  src/
    app/                # App shell, routing, providers
    features/
      auth/             # Login, register, password reset
      dashboard/
      invoices/
      webhooks/
      apikeys/
      team/
      settings/
      admin/
    components/
      ui/               # shadcn/ui primitives (Button, Card, Table, Modal...)
      charts/
      layout/           # Sidebar, Topbar, PageHeader
    lib/
      api-client/       # generated types + fetch wrapper
      auth/             # token storage, refresh logic
      validation/       # zod schemas
      hooks/            # useInvoices, useAuth, etc.
    styles/
      theme.css         # CSS variables = brand palette
    types/
  public/
  index.html
  package.json
```

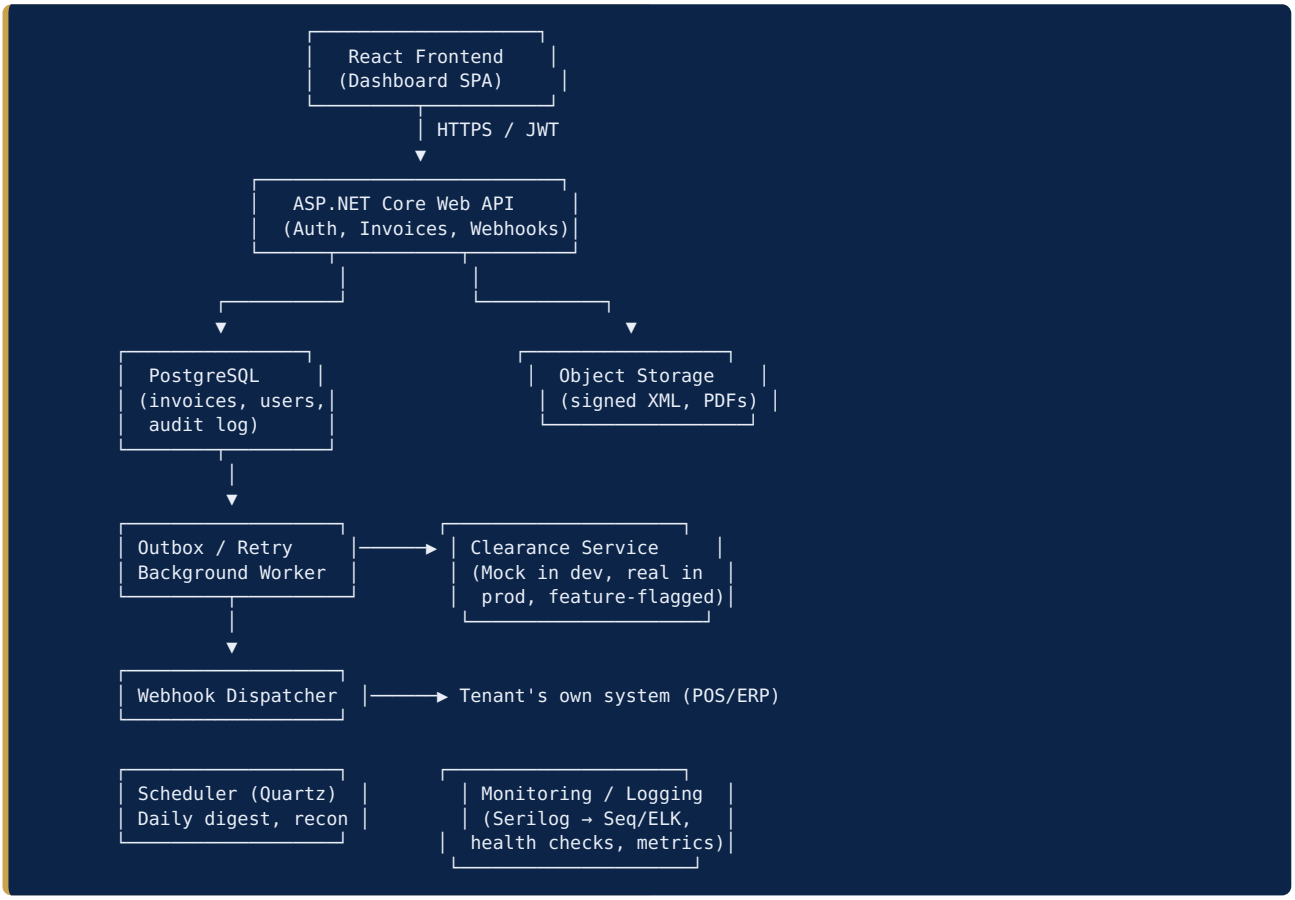
# 9. Backend Architecture

- **Backend language:** C# (.NET 8).
- **Framework:** ASP.NET Core Web API.
- **Architecture pattern:** Clean Architecture (Domain → Application → Infrastructure → API), with CQRS-lite (MediatR for command/query handlers) to keep controllers thin.
- **Layers:**
- **API layer** — controllers, request/response DTOs, auth middleware, OpenAPI generation.
- **Application layer** — use-case handlers (`SubmitInvoiceCommand`, `GetInvoicesQuery`), validation (FluentValidation), mapping (AutoMapper or manual mappers).
- **Domain layer** — entities (`Invoice`, `ClearanceAttempt`, `Tenant`), value objects (`TaxId`, `Money`), domain events (`InvoiceCleared`), business rules independent of any framework.
- **Infrastructure layer** — EF Core `DbContext` + repositories, the clearance-service HTTP client, the UBL 2.1 XML mapper, the digital-signature service, object storage client, email/webhook senders.
- **Services:** `InvoiceSubmissionService`, `UblMappingService`, `SigningService`, `ClearanceClient` (wraps the external/mock API), `RetryQueueService`, `WebhookDispatchService`, `PdfRenderingService`, `AuditLogService`.
- **Controllers:** `InvoicesController`, `AuthController`, `WebhooksController`, `ApiKeysController`, `TeamController`, `TenantsController`, `AnalyticsController`, `AdminController` (System Operator only).
- **Domain layer contents:** Rich entities with invariants enforced in constructors/methods (e.g., an `Invoice` cannot transition from `CLEARED` back to `DRAFT`), not anemic data bags.
- **Data-access layer:** EF Core with PostgreSQL provider (see §22 for SQL Server vs. PostgreSQL note), repository interfaces defined in Domain, implemented in Infrastructure (Dependency Inversion).
- **Integration layer:** `IClearanceClient` interface with two implementations — `MockClearanceClient` (default, deterministic test double simulating the government service) and `LiveClearanceClient` (real endpoint, feature-flagged off by default).
- **Workers:** A hosted background service (`IHostedService`) running the retry loop; a separate hosted service for the webhook dispatch queue; both backed by a lightweight persistent queue (PostgreSQL-backed outbox table for MVP; can graduate to a real message broker later).
- **Queues:** Implemented as an **Outbox pattern** table (`OutboxMessage`) written in the same DB transaction as the state change, guaranteeing at-least-once delivery without a separate broker dependency for the MVP.
- **Schedulers: Quartz.NET** (or `IHostedService` + `PeriodicTimer`) for the daily digest and nightly reconciliation job.

**Component communication:** Controllers dispatch MediatR commands/queries → Application handlers orchestrate Domain entities + call Infrastructure interfaces → Infrastructure talks to the DB, the (mock) clearance API, object storage, and email/webhook transports → Domain events raised by entities are published after the DB transaction commits, triggering side effects (webhook dispatch, audit log write) via MediatR notification handlers — keeping the core submission logic decoupled from "what happens after."

---

# 10. Complete System Architecture




Frontend → API (HTTPS, JWT bearer) → Authentication middleware validates token → Backend service layer (Application) executes business logic against Domain → Data-access layer persists to PostgreSQL → On state change, an Outbox event is written in the same transaction → Background worker picks up outbox events → dispatches to the (mock) Clearance Service and/or the Webhook Dispatcher → Object storage holds signed documents and rendered PDFs → Scheduler triggers periodic jobs independently → All layers emit structured logs and metrics to the monitoring stack, with health-check endpoints exposed for uptime monitoring.

# 11. Database Design

`tenants`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| legal_name | varchar(255) | required |
| tax_id | varchar(50) | required, unique |
| address | text | nullable |
| default_currency | varchar(3) | default 'JOD' |
| plan | varchar(20) | default 'trial' |
| created_at | timestamptz | required |
| deactivated_at | timestamptz | nullable |

`users`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| tenant_id | uuid | FK → tenants.id, required |
| email | varchar(255) | required, unique |
| password_hash | varchar(255) | required |
| role | varchar(20) | required (`TenantAdmin`, `FinanceUser`, `Developer`, `SystemOperator`) |
| email_verified | boolean | default false |
| status | varchar(20) | default 'pending' (`pending`, `active`, `deactivated`) |
| created_at | timestamptz | required |
| index | | (tenant_id, email) |

`api_keys`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| tenant_id | uuid | FK → tenants.id |
| name | varchar(100) | required |
| key_hash | varchar(255) | required (only the hash is stored) |
| key_prefix | varchar(12) | required (shown in UI for identification) |
| created_at | timestamptz | required |
| last_used_at | timestamptz | nullable |
| revoked_at | timestamptz | nullable |

`invoices`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| tenant_id | uuid | FK → tenants.id, required, indexed |
| external_ref | varchar(100) | nullable (client's own invoice number) |
| buyer_name | varchar(255) | nullable (per JoFotara rule, optional under JOD 10,000) |
| buyer_tax_id | varchar(50) | nullable |
| seller_snapshot | jsonb | required (denormalized seller info at time of issue) |
| currency | varchar(3) | required |
| subtotal | numeric(14,3) | required |
| tax_total | numeric(14,3) | required |
| grand_total | numeric(14,3) | required |
| status | varchar(20) | required (`DRAFT`, `SUBMITTED`, `CLEARED`, `FAILED`, `NEEDS_ATTENTION`) |
| ubl_document_ref | varchar(255) | nullable (object storage key) |
| qr_payload | text | nullable |
| pdf_ref | varchar(255) | nullable (object storage key) |
| created_at | timestamptz | required |
| cleared_at | timestamptz | nullable |
| index | | (tenant_id, status), (tenant_id, created_at) |

**`invoice_lines`**

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| invoice_id | uuid | FK → invoices.id, required |
| description | varchar(500) | required |
| quantity | numeric(10,3) | required |
| unit_price | numeric(14,3) | required |
| tax_rate | numeric(5,2) | required |
| line_total | numeric(14,3) | required |

**`clearance_attempts`**

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| invoice_id | uuid | FK → invoices.id, required, indexed |
| attempt_uuid | uuid | required, unique (fresh UUID per attempt) |
| submitted_at | timestamptz | required |
| response_code | varchar(20) | nullable |
| success | boolean | required |
| error_detail | text | nullable |
| raw_response | jsonb | nullable |

**`outbox_messages`**

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| type | varchar(50) | required (`RetryInvoice`, `DispatchWebhook`, `SendEmail`) |
| payload | jsonb | required |
| status | varchar(20) | default 'pending' |
| attempts | int | default 0 |
| next_attempt_at | timestamptz | required |
| created_at | timestamptz | required |

`webhook_endpoints`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| tenant_id | uuid | FK → tenants.id, required |
| url | varchar(500) | required |
| signing_secret | varchar(255) | required |
| subscribed_events | varchar(255)[] | required |
| active | boolean | default true |

`webhook_deliveries`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| endpoint_id | uuid | FK → webhook_endpoints.id |
| event_type | varchar(50) | required |
| payload | jsonb | required |
| response_status | int | nullable |
| delivered_at | timestamptz | nullable |
| attempt_count | int | default 0 |

`audit_logs`

| Field | Type | Constraints |
|---|---|---|
| id | uuid | PK |
| tenant_id | uuid | FK → tenants.id, indexed |
| actor_user_id | uuid | nullable (null = system action) |
| action | varchar(100) | required |
| entity_type | varchar(50) | required |
| entity_id | uuid | required |
| before_state | jsonb | nullable |
| after_state | jsonb | nullable |
| created_at | timestamptz | required, indexed |

## Relationships

- `tenants` 1-to-many `users`, `api_keys`, `invoices`, `webhook_endpoints`, `audit_logs`.
- `invoices` 1-to-many `invoice_lines`, `clearance_attempts`.

- `webhook_endpoints` 1-to-many `webhook_deliveries`.
- No many-to-many relationships in the MVP domain; a future "shared branches across tenants" feature would introduce one via a `tenant_branches` join table.

---

# 12. API Design

All endpoints are versioned under `/api/v1`. Authentication: `Authorization: Bearer <JWT>` for dashboard users, `X-API-Key: <key>` for machine clients. Responses follow a consistent envelope: `{ "data": ..., "error": null }` or `{ "data": null, "error": { "code": "...", "message": "..." } }`.

### Auth Module

| Method | URL | Purpose | Auth | Request | Response | Success | Errors |
|---|---|---|---|---|---|---|---|
| POST | /auth/register | Create tenant + admin user | None | `{legalName, taxId, email, password}` | `{userId, tenantId}` | 201 | 400 validation, 409 duplicate |
| POST | /auth/login | Authenticate | None | `{email, password}` | `{accessToken, expiresIn}` + refresh cookie | 200 | 401 invalid credentials, 423 locked |
| POST | /auth/refresh | Rotate access token | Refresh cookie | — | `{accessToken}` | 200 | 401 expired/invalid |
| POST | /auth/logout | Revoke refresh token | Refresh cookie | — | — | 204 | — |
| POST | /auth/verify-email | Confirm email | None (token in body) | `{token}` | — | 200 | 400 invalid/expired |
| POST | /auth/forgot-password | Request reset | None | `{email}` | generic message | 200 | — |
| POST | /auth/reset-password | Set new password | None (token in body) | `{token, newPassword}` | — | 200 | 400 invalid/expired |

### Invoices Module

| Method | URL | Purpose | Auth | Permissions | Request | Response | Success | Errors |
|---|---|---|---|---|---|---|---|---|
| POST | /invoices | Submit a new invoice | API key or JWT | Developer, FinanceUser | Invoice payload (buyer, lines, totals) | Invoice object w/ status | 202 (async clearance) | 400 schema, 422 business-rule violation |
| GET | /invoices | List invoices | JWT/API key | any tenant role | Query: status, dateFrom, dateTo, page, pageSize | Paginated invoice list | 200 | 400 bad query |
| GET | /invoices/{id} | Get invoice detail | JWT/API key | any tenant role | — | Invoice + lines + clearance history | 200 | 404 |
| POST | /invoices/{id}/resubmit | Manually retry a failed invoice | JWT/API key | FinanceUser+ | — | Updated invoice | 202 | 404, 409 (already cleared) |
| GET | /invoices/{id}/pdf | Download cleared PDF | JWT/API key | any tenant role | — | binary/pdf | 200 | 404, 409 (not yet cleared) |
| GET | /invoices/export | Export CSV of invoices | JWT | FinanceUser+ | Query filters | CSV stream | 200 | 400 |

### Webhooks Module

| Method | URL | Purpose | Auth | Request | Response |
|---|---|---|---|---|---|
| GET | /webhooks | List configured endpoints | JWT | — | list |
| POST | /webhooks | Register endpoint | JWT (Developer/Admin) | `{url, events[]}` | endpoint + secret |
| DELETE | /webhooks/{id} | Remove endpoint | JWT (Developer/Admin) | — | 204 |
| POST | /webhooks/{id}/test | Send a test event | JWT | — | delivery result |
| GET | /webhooks/{id}/deliveries | Delivery log | JWT | — | list |

## API Keys Module

| Method | URL | Purpose | Auth | Request | Response |
|---|---|---|---|---|---|
| GET | /api-keys | List keys (prefix only) | JWT (Admin) | — | list |
| POST | /api-keys | Generate new key | JWT (Admin) | `{name}` | `{key}` (shown once) |
| DELETE | /api-keys/{id} | Revoke key | JWT (Admin) | — | 204 |

## Team Module

| Method | URL | Purpose | Auth | Request | Response |
|---|---|---|---|---|---|
| GET | /team | List members | JWT | — | list |
| POST | /team/invite | Invite user | JWT (Admin) | `{email, role}` | invite record |
| PATCH | /team/{userId} | Change role/status | JWT (Admin) | `{role?, status?}` | updated user |
| DELETE | /team/{userId} | Remove user | JWT (Admin) | — | 204 |

## Analytics Module

| Method | URL | Purpose | Auth | Response |
|---|---|---|---|---|
| GET | /analytics/summary | Cleared/failed counts, VAT totals | JWT | summary object |
| GET | /analytics/trend | Daily time series | JWT | array of {date, cleared, failed} |

## System / Health

| Method | URL | Purpose | Auth |
|---|---|---|---|
| GET | /health | Liveness check | None |
| GET | /health/ready | Readiness (DB, storage reachable) | None |

# 13. Authentication & Authorization

- **Registration:** Email + password + company details; password hashed with **Argon2id** (memory-hard, resistant to GPU cracking) before storage; never logged, never returned in any response.
- **Login:** Credential check against Argon2id hash → on success, issue a short-lived **JWT access token** (15 minutes, signed with RS256, contains `sub`, `tenantId`, `role`, `exp`) and a long-lived **refresh token** (7 days, opaque random value, stored hashed in DB, delivered as httpOnly/Secure/SameSite=Strict cookie).
- **Password hashing:** Argon2id, tuned parameters (memory ≥ 64MB, iterations ≥ 3) per current OWASP guidance.
- **JWT/session strategy:** Stateless access tokens for API authorization (fast, no DB hit per request); stateful refresh tokens (DB-backed, revocable) for session control — gives the best of both: performance and the ability to force-logout a compromised account.
- **Refresh tokens:** Rotated on every use (old one invalidated, new one issued) to detect token theft (reuse of an old refresh token triggers full session revocation for that user).
- **Password reset:** Signed, time-limited (30 min) token emailed via a link; resetting the password revokes all existing refresh tokens for that user.
- **Email verification:** Required before first login to a production tenant; signed token, 24-hour expiry, resend available.
- **Roles:** `TenantAdmin` (full tenant control), `FinanceUser` (view/manage invoices, cannot manage team/keys), `Developer` (manage API keys/webhooks, read invoices, cannot manage team), `SystemOperator` (platform-wide, read-only across tenants, internal only).
- **Permissions:** Enforced via a policy-based authorization layer in ASP.NET Core (`[Authorize(Policy = "CanManageTeam")]`), mapped from roles to a permission matrix so new roles can be added without rewriting checks scattered across controllers.
- **Admin privileges:** `TenantAdmin` can manage users/roles/keys/webhooks/settings within their own tenant only; `SystemOperator` has cross-tenant read access for support but cannot modify tenant financial data (enforced at the query layer, not just the UI).
- **Tenant isolation:** Every query is scoped by `tenant_id` derived from the authenticated principal (JWT claim or API key lookup), never from a client-supplied parameter — enforced centrally via a query filter (EF Core global query filters) so it cannot be forgotten in a new endpoint.

# 14. Security

- **OWASP considerations:** The design is checked against the OWASP Top 10: Broken Access Control (mitigated by tenant-scoped query filters + policy-based authZ), Cryptographic Failures (Argon2id hashing, TLS everywhere, signed documents), Injection (parameterized queries via EF Core, no raw SQL string concatenation), Insecure Design (threat-modeled retry/idempotency flows), Security Misconfiguration (locked-down CORS, security headers), Vulnerable Components (automated dependency scanning in CI), Auth Failures (rate-limited login, Argon2id, MFA-ready design), Data Integrity Failures (signed webhook payloads, audit log), Logging Failures (structured audit + security logging), SSRF (webhook URL validation against private IP ranges before dispatch).
- **Input validation:** FluentValidation on every command DTO server-side (never trust client-side validation alone); strict schema validation on the invoice payload (types, required fields, max lengths, numeric bounds) before any UBL mapping is attempted.
- **SQL injection:** Fully mitigated by EF Core's parameterized queries; no dynamic SQL string building anywhere in the codebase; a CI lint rule flags any raw `ExecuteSqlRaw` usage for manual review.
- **XSS:** React escapes output by default; any rendered HTML (none expected in this domain) would go through a sanitizer; strict `Content-Security-Policy` header set on all API responses and the frontend.
- **CSRF:** Refresh-token cookie is `SameSite=Strict`; state-changing API requests require a custom header (`X-Requested-With`) that simple cross-site forms cannot set, plus the JWT bearer pattern for the actual authorization (CSRF is primarily a cookie-auth problem; bearer-token auth on the access token largely sidesteps it, with the refresh cookie hardened as above).
- **Authentication attacks:** Login rate-limited (5 attempts per 15 minutes per IP+email combo, exponential lockout); generic error messages prevent user enumeration; refresh-token rotation detects reuse/theft.
- **Rate limiting:** Per-API-key and per-IP rate limiting (ASP.NET Core `RateLimiter` middleware), tiered by plan (e.g., 100 req/min for trial, higher for paid), 429 responses with `Retry-After` header.
- **Secrets management:** All secrets (JWT signing key, DB connection string, digital signing key, webhook signing secrets) held in environment variables locally and a managed secret store in production (Azure Key Vault / AWS Secrets Manager); never committed to source control (`.env` is gitignored, `.env.example` provided).
- **Encryption:** TLS 1.2+ enforced for all traffic (HSTS header); sensitive fields (tax IDs) encrypted at rest using column-level encryption where the database provider supports it; signing keys stored in the secret manager, not the database.
- **HTTPS:** Enforced end-to-end; HTTP requests redirected; HSTS with a 1-year max-age.
- **API security:** All endpoints require authentication except `/health` and the public auth endpoints; API keys are hashed (never stored plaintext) with only a short prefix retained for UI identification; keys can be scoped/revoked instantly.
- **Access control:** Enforced at three layers — route-level policy attributes, service-level tenant-scoping, and database-level query filters — defense in depth rather than relying on a single check.
- **Audit logs:** Every state-changing action (invoice submission, resubmission, user role change, key generation/revocation) writes an immutable `audit_logs` row capturing actor, before/after state, and timestamp; audit logs are append-only (no update/delete permission at the DB role level).
- **File-upload security:** N/A for MVP (no user file uploads); if bulk CSV import (v1.1) is added, files are size-limited, type-validated (MIME + extension), scanned, and processed in an isolated worker, never executed.
- **Prompt injection:** N/A — ClearVoice's MVP has no AI component (the optional "error explainer" LLM feature, if built, would treat all invoice/error data as untrusted input and never allow the model's output to alter invoice data directly — see the AI Architecture section of the InsightDesk AI document for the shared pattern).

# 15. Automation & Event-Driven Workflows

- **Webhooks:** Tenants register a URL + choose event types (`invoice.cleared`, `invoice.failed`, `invoice.needs_attention`, `invoice.resubmitted`). Payloads are HMAC-SHA256 signed with the tenant's webhook secret (`X-ClearVoice-Signature` header) so receivers can verify authenticity.
- **Queues:** MVP uses a **transactional outbox** pattern — an `outbox_messages` row is written in the same DB transaction as the domain state change, guaranteeing the event is never lost even if the process crashes immediately after. A background poller claims and processes outbox rows.
- **Workers:** `RetryWorker` (clearance retries), `WebhookDispatchWorker` (delivers webhook events), both implemented as `IHostedService` background loops polling the outbox with `SELECT ... FOR UPDATE SKIP LOCKED` semantics to allow safe horizontal scaling of workers later.
- **Retry logic:** Exponential backoff with jitter: attempt 2 after 1 min, attempt 3 after 5 min, attempt 4 after 15 min, attempt 5 after 1 hour, then `NEEDS_ATTENTION`. Each attempt gets a **fresh UUID** (`attempt_uuid`) so the clearance service never sees a duplicate submission ID.
- **Scheduled jobs:** Daily compliance digest (07:00 Amman time), nightly reconciliation sweep (cross-checks local `CLEARED` status against the clearance service where a status-check endpoint exists), weekly stale-outbox alert (ops-facing, flags any outbox row older than 24h that hasn't been processed).
- **Event handling:** Domain entities raise events (`InvoiceCleared`, `InvoiceFailed`) which MediatR publishes as notifications *after* the DB transaction commits — this guarantees the event only fires for state that's actually durable, avoiding "phantom" webhook deliveries for a rolled-back transaction.
- **Idempotency:** All webhook payloads include an `eventId`; receivers are instructed to deduplicate on this ID. All invoice-submission API calls accept an optional client-supplied `Idempotency-Key` header — if the same key is seen twice, the cached original response is returned rather than re-processing.
- **Failure recovery:** If the clearance service itself is down (not a validation failure), attempts are classified as `TRANSIENT_FAILURE` and retried more aggressively (shorter backoff) than `VALIDATION_FAILURE`, which requires human correction and stops auto-retrying after 1 attempt (retrying a malformed invoice five times wastes cycles).
- **Dead-letter handling:** Outbox rows that exceed max attempts move to a `dead` status, surfaced in the Admin — Platform Overview screen for manual operator intervention, rather than being silently dropped.

---

# 16. AI Architecture

**Note:** ClearVoice's core MVP deliberately contains **no AI** — compliance software must be deterministic, auditable, and explainable, and forcing an LLM into the clearance pipeline itself would undermine trust. The one optional, clearly-scoped AI feature below is additive and never touches the legally-binding submission data.

- **Why AI is needed (optional feature only):** Clearance rejection error codes/messages returned by tax authorities are often terse and technical (e.g., `ERR_BUYER_TIN_INVALID`). Non-technical business owners struggle to know what to actually fix.
- **Which task AI performs:** "Explain this rejection in plain language and suggest the likely fix" — a translation/summarization task, not a decision-making task.
- **Model/provider options:** A small, cheap instruction-tuned model (e.g., a lightweight hosted LLM API) is sufficient — this is not a reasoning-heavy task.
- **Prompt strategy:** A tightly-constrained system prompt instructing the model to (a) only reference the provided error code/message and invoice field names, (b) never invent tax rules not present in the input, (c) always end with "Please verify before resubmitting."
- **Input:** The structured error code, the raw error message from the clearance attempt, and the relevant invoice field(s) — nothing else.
- **Output:** A short plain-language explanation (2–3 sentences) shown as a tooltip/panel next to the rejected invoice.
- **Context:** No retrieval needed for MVP — the input is small and self-contained; if rejection-code documentation grows large, a small RAG lookup over an internal error-code glossary could be added later.
- **RAG:** Not required for MVP; noted as a future enhancement only if the error-code glossary becomes too large for the prompt.
- **Embeddings / vector database:** Not used in ClearVoice.
- **Guardrails:** The model's output is display-only text; it can never modify invoice data, trigger a resubmission, or bypass validation. Output is passed through a simple keyword filter to strip anything resembling a change instruction before rendering.
- **Human approval:** The business owner reads the explanation and manually edits/resubmits — the AI never acts autonomously.
- **Hallucination mitigation:** Constrained prompt + narrow input scope + display-only output + a persistent "AI-generated explanation, please verify" disclaimer.
- **Cost controls:** This feature is called only on-demand (user clicks "Explain this error"), not automatically on every failure, keeping API usage proportional to actual need; responses are cached per error-code+message combination to avoid redundant calls.

---

# 17. External Integrations

| Integration | Purpose | API | Auth | Data sent | Data received | Failure behavior |
|---|---|---|---|---|---|---|
| National Clearance Service (mocked in dev, real in prod) | Legally clear invoices | REST/SOAP (per authority spec) | Client ID + Secret | Signed UBL 2.1 XML invoice | Signed copy + QR code, or error | Retried per §15; classified transient vs. validation |
| Transactional email provider (e.g., SendGrid/Postmark) | Verification, reset, digests | REST | API key | Recipient, template data | Delivery status | Queued retry; failure logged, does not block core flow |
| Object storage (S3-compatible / Azure Blob) | Store signed XML + PDFs | REST/SDK | Access key / managed identity | Binary documents | Storage URL/key | Retried; if persistent, invoice marked "document pending" without blocking clearance status |
| Tenant webhook endpoints | Notify client systems | REST (tenant-provided URL) | HMAC signature | Invoice status event payload | HTTP status code | Retried per own backoff policy; disabled after repeated hard failures (with tenant notification) |
| Optional: LLM provider (§16 feature only) | Explain rejection errors | REST | API key | Error code + message + field names | Plain-language text | Feature gracefully hides if provider unavailable; never blocks core invoice flow |
| Optional (future): ERP connectors (Odoo, QuickBooks) | Pull invoice data automatically | REST/SDK per platform | OAuth2 | Invoice data | Confirmation | Falls back to manual API submission |

# 18. Performance & Scalability

- **Expected load (portfolio-scale):** Tens to low hundreds of invoices per day per demo tenant — trivial load; the architecture is designed to demonstrate correctness and scalability *thinking*, not to actually handle enterprise volume on a portfolio budget.
- **Database optimization:** Composite indexes on `(tenant_id, status)` and `(tenant_id, created_at)` for the two most common query patterns (dashboard filtering, chronological listing); numeric fields use fixed-precision `numeric` types (never floating point) for monetary correctness.
- **Indexing:** Covered above; additionally a unique index on `attempt_uuid` to guarantee no duplicate clearance submissions, and on `(tenant_id, email)` for login lookups.
- **Caching:** Read-heavy, rarely-changing data (tenant settings, webhook endpoint config) cached in-memory (`IMemoryCache`) with short TTL; analytics summary queries cached for 60 seconds to avoid recomputing aggregates on every dashboard load.
- **Connection pooling:** Npgsql's built-in connection pooling configured with sane min/max pool sizes; EF Core `DbContext` scoped per-request to avoid leaking connections.
- **Async processing:** All I/O-bound work (clearance API calls, email, webhook delivery, PDF rendering) is fully async (`async`/`await` throughout, no blocking `.Result` calls) and offloaded to background workers rather than the request thread wherever possible, keeping the submission endpoint responsive.
- **Horizontal scaling:** The API is stateless (JWT-based auth, no server-side session), so it can run as multiple container replicas behind a load balancer; the outbox-based worker pattern with `SKIP LOCKED` allows multiple worker instances to safely process the same queue without duplicate work.
- **Load balancing:** For the "better production-style deployment" (§22), a managed load balancer/App Service auto-scale rule distributes traffic across API instances.
- **CDN:** The React frontend's static build is served via a CDN (e.g., Azure Static Web Apps or Cloudflare) for fast global asset delivery; not relevant to the backend API.
- **MVP vs. at-scale:** For MVP: single API instance, single Postgres instance, single worker process — entirely sufficient and honestly demonstrable on a portfolio deployment. At scale, the next steps (documented but not built) would be: read replicas for analytics queries, a real message broker (RabbitMQ/SQS) replacing the outbox table, Redis for distributed caching, and horizontal worker scaling.

# 19. Testing Strategy

- **Unit tests:** Domain logic (invoice state transitions, UBL mapping rules, retry backoff calculation) tested in isolation with **xUnit**, no database or network dependency — these should be the fastest and most numerous tests.
- **Integration tests:** API endpoints tested against a real (containerized, ephemeral) PostgreSQL instance using **Testcontainers**, verifying the full request → handler → DB → response path, including tenant-isolation query filters.
- **API tests:** Contract-level tests asserting the OpenAPI spec matches actual responses (prevents frontend/backend drift); a Postman/Newman or `dotnet` HTTP-file-based smoke suite runnable against a live deployment.
- **Database tests:** Migration tests (apply all migrations to a fresh DB, verify no errors); constraint tests (verify unique constraints, FK cascades behave as designed).
- **Authentication tests:** Login lockout after N failed attempts, refresh-token rotation/reuse detection, tenant-isolation (user from Tenant A cannot fetch Tenant B's invoice by guessing an ID — this is arguably the single most important test in the whole suite).
- **Security tests:** Automated dependency vulnerability scanning in CI (`dotnet list package --vulnerable`, `npm audit`); a small set of "negative" tests asserting SQL-injection-style and XSS-style payloads are safely rejected/escaped.
- **End-to-end tests: Playwright** covering the critical user journeys — register → verify → onboard → submit invoice via API → see it cleared on dashboard → download PDF; and the failure path — submit invalid invoice → see it in Needs Attention → resubmit → see it clear.
- **Load tests:** A lightweight **k6** script simulating concurrent invoice submissions to validate the retry worker and outbox pattern don't create duplicate clearance attempts under concurrency — useful primarily as an interview talking point about correctness under load, not as a production capacity test.
- **What to test first (priority order):** (1) tenant isolation and auth, (2) invoice submission + UBL mapping correctness, (3) retry/idempotency logic (fresh UUID per attempt, no duplicate submissions), (4) webhook signing/delivery, (5) everything else.

---

# 20. Logging & Monitoring

- **Application logs:** Structured JSON logging via **Serilog**, with correlation IDs propagated from the incoming HTTP request through every downstream service call and background job, so a single invoice's entire journey can be traced by one ID.
- **Error logs:** Unhandled exceptions captured centrally by global exception-handling middleware, logged with full context (tenant ID, request path, correlation ID) but with sensitive fields (passwords, secrets) redacted by a Serilog destructuring policy.
- **Audit logs:** Distinct from application logs — the `audit_logs` table (§11) is a business-facing, immutable, queryable record of who-did-what, separate from operational/debug logging.
- **Request tracing:** OpenTelemetry instrumentation on the API, exporting traces that show the full path of a request including the outbound call to the (mock) clearance service — makes latency bottlenecks visible.
- **Performance metrics:** Key metrics exported (clearance success rate, average clearance latency, outbox queue depth, retry-worker lag) via `dotnet-counters`/OpenTelemetry metrics, scrapeable by Prometheus.
- **Health checks:** `/health` (liveness — is the process running) and `/health/ready` (readiness — can it reach the database and object storage) implemented via ASP.NET Core's built-in `HealthChecks` middleware.
- **Uptime monitoring:** An external uptime checker (e.g., UptimeRobot or a cloud provider's monitor) pings `/health` every minute on the public deployment, with alerting on downtime — cheap and demonstrable on a portfolio budget.
- **Alerting:** For MVP/portfolio scope, alerting is email-based (triggered by the uptime monitor and by the "stale outbox" scheduled job in §15); the architecture notes where this would graduate to PagerDuty/Slack integration at production scale.

# 21. DevOps

- **Git strategy:** Trunk-based development with short-lived feature branches (`feature/invoice-retry-worker`), branch names prefixed by type (`feature/`, `fix/`, `chore/`).
- **Branch structure:** `main` (always deployable, protected, requires passing CI + 1 review), feature branches merged via PR and squash-merged to keep history clean.
- **Pull requests:** PR template requiring a description, linked issue, and a checklist (tests added, docs updated, no secrets committed); CI must pass before merge is allowed.
- **Environment variables:** `.env.example` checked into the repo documenting every required variable (DB connection string, JWT signing key, storage credentials, clearance-service mode) with placeholder values; real values only in local `.env` (gitignored) or the platform's secret store in deployed environments.
- **Docker:** Multi-stage Dockerfile for the API (build stage with full SDK, runtime stage with slim ASP.NET runtime image); a separate Dockerfile for the frontend (build stage with Node, serve stage with a static file server/nginx).
- **Docker Compose:** A single `docker-compose.yml` spinning up API + PostgreSQL + frontend + mock-clearance-service + a mail-catcher (e.g., MailHog) for local development — one command (`docker compose up`) gets a reviewer to a fully working local instance.
- **CI/CD:** GitHub Actions pipeline: on every PR — restore, build, run unit + integration tests, lint, dependency-vulnerability scan; on merge to `main` — additionally build and push Docker images, then deploy to the staging environment automatically, with production deployment gated behind a manual approval step.
- **GitHub Actions:** Separate workflow files for `ci.yml` (test/lint on PR) and `deploy.yml` (build/push/deploy on merge to main), keeping fast feedback loops separate from slower deployment steps.
- **Staging:** A long-lived staging environment (same architecture as production, smaller resources) that every merge to `main` deploys to automatically — this is the environment shown in the live demo.
- **Production:** A manually-gated promotion from staging (for a portfolio project, staging *is* effectively production/demo; the distinction is documented as the intended real-world pattern).
- **Rollback strategy:** Docker images are tagged with the Git commit SHA; rollback is redeploying the previous image tag; database migrations are written to be backward-compatible for at least one release (additive changes preferred over destructive ones) so a code rollback doesn't require a matching DB rollback.

---

# 22. Cloud Deployment

**Recommendation: Microsoft Azure**, because it pairs naturally with the ASP.NET Core stack (first-class tooling, `dotnet` CLI integration, generous free-tier credits for students/new accounts) and is a stack Amman's .NET-heavy employers (ProgressSoft, Optimiza, Aspire, Minerets) will recognize immediately.

### Cheapest Portfolio Deployment

- **Application hosting:** Azure App Service (Free or Basic B1 tier) for the ASP.NET Core API, or a single small VM running the Docker Compose stack.
- **Database hosting:** Azure Database for PostgreSQL — Flexible Server, Burstable tier (smallest SKU) — swapping SQL Server for PostgreSQL here (vs. the SQL Server used in early local development) proves stack portability, a genuine interview talking point.
- **Frontend hosting:** Azure Static Web Apps (free tier) serving the built React app, with a built-in CDN.
- **Object storage:** Azure Blob Storage (pay-as-you-go, negligible cost at portfolio volume) for signed XML/PDFs.
- **Secrets:** Azure Key Vault (free tier covers portfolio-scale secret operations).
- **Monitoring:** Azure Monitor / Application Insights free tier for basic logs and metrics.
- **Background workers:** Run as a second process within the same App Service Plan (or a low-cost Azure Container App) rather than a separate expensive compute resource.

### Better Production-Style Deployment

- **Application hosting:** Azure Container Apps or AKS (Kubernetes) for the API and workers, enabling auto-scaling and rolling deployments.
- **Database hosting:** Azure Database for PostgreSQL — General Purpose tier with read replicas for analytics queries.
- **Object storage:** Blob Storage with lifecycle policies (archive old documents to cool/cold tiers).
- **Secrets:** Key Vault with managed-identity-based access (no credentials in app config at all).
- **Monitoring:** Full Application Insights + Log Analytics workspace, with alert rules feeding into a Teams/Slack channel.
- **CDN:** Azure Front Door in front of both the static frontend and the API for global edge caching and DDoS protection.
- **Message broker upgrade:** Replace the outbox-table pattern with Azure Service Bus for the retry/webhook queues at real production volume.

(AWS is a fully viable alternative — App Runner/ECS, RDS for PostgreSQL, S3, Secrets Manager, CloudWatch map one-to-one onto the above; Azure is recommended here specifically for its natural fit with the .NET stack and the local employer landscape.)

---

# 23. GitHub Repository

```
clearvoice/
├── src/
│   ├── ClearVoice.Api/              # Controllers, Program.cs, middleware
│   ├── ClearVoice.Application/      # Command/query handlers, validators
│   ├── ClearVoice.Domain/           # Entities, value objects, domain events
│   ├── ClearVoice.Infrastructure/   # EF Core, clearance client, storage, email
│   └── ClearVoice.Workers/          # Retry worker, webhook dispatcher, scheduler
├── frontend/
│   ├── src/
│   ├── public/
│   └── package.json
├── tests/
│   ├── ClearVoice.UnitTests/
│   ├── ClearVoice.IntegrationTests/
│   └── ClearVoice.E2ETests/          # Playwright specs
├── docs/
│   ├── architecture.md
│   ├── architecture-diagram.png
│   ├── api-examples/
│   └── ubl-mapping-notes.md
├── infra/
│   ├── docker/
│   │   ├── Dockerfile.api
│   │   └── Dockerfile.frontend
│   ├── docker-compose.yml
│   └── azure/                        # Bicep/Terraform templates (optional)
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── deploy.yml
├── .env.example
├── .gitignore
├── LICENSE
└── README.md
```

# 24. README

**Exact section outline:** 1. **ClearVoice** — one-line tagline + badges (build status, license) 2. **Live Demo** — public URL + demo credentials 3. **The Problem** — short paragraph on Jordan's mandatory e-invoicing regulation, with a link to source 4. **Screenshots** — dashboard, invoice detail, needs-attention queue 5. **Architecture** — the diagram from §10, brief walkthrough 6. **Tech Stack** — table of technologies + why each was chosen 7. **Features** — bullet summary of MVP features 8. **Getting Started** — prerequisites, `git clone`, `docker compose up` 9. **Environment Setup** — link to `.env.example`, explanation of each variable 10. **API Documentation** — link to hosted Swagger/OpenAPI UI, plus a curl example 11. **Database Schema** — link to `docs/architecture.md` ERD 12. **Testing** — how to run unit/integration/e2e tests 13. **Security** — summary of the measures in §14, responsible-disclosure note 14. **Deployment** — summary of §22, link to CI/CD workflow files 15. **Roadmap** — the MVP → v1.1 → v2 feature progression from §4 16. **License**

# 25. Demo Experience

**Intended recruiter journey:** 1. Recruiter opens the GitHub repository → sees a polished README with a badge row, screenshots, and a one-paragraph problem statement that immediately signals "this person understands a real regulatory domain." 2. Clicks the **Live Demo** link → lands on the ClearVoice dashboard, already logged into a seeded demo tenant (no signup friction). 3. Sees the dashboard populated with realistic seeded data: 40+ invoices, a mix of cleared/failed/needs-attention statuses, a trend chart with visible variance. 4. Clicks into a cleared invoice → sees the QR-stamped PDF and the full clearance-attempt timeline. 5. Clicks into a "Needs Attention" invoice → sees the plain-language error, clicks **Resubmit** → watches the status update in real time (demonstrating the retry/webhook pipeline actually works, not just looks nice). 6. Clicks **API Docs** in the nav → sees the live Swagger UI, tries a `GET /invoices` call with the demo API key provided in the README. 7. Returns to GitHub, watches the embedded 2-minute Loom video walking through the architecture diagram and the "why" behind the design decisions. 8. Optionally views the architecture diagram image directly in `/docs`.

**Demo account details to provide:** A read-only demo login (email/password shown in the README and on the live login page itself as a "Try Demo Account" button — zero friction), and a read-only demo API key for the Swagger UI.

---

# 26. LinkedIn Presentation

**Project title:** ClearVoice — E-Invoicing Compliance Gateway

**Short LinkedIn description:** "Built ClearVoice, a middleware API that generates, signs, and clears e-invoices against Jordan's mandatory national e-invoicing system in real time — with automatic retries, webhooks, and full audit trails."

**Full LinkedIn description:** "Since April 2025, every business in Jordan must clear invoices through the national e-invoicing system before they're legally valid — non-compliant invoices aren't even tax-deductible for the buyer. Most small businesses have no way to integrate this correctly.

I built ClearVoice to solve it: a Clean-Architecture ASP.NET Core Web API that maps invoices to UBL 2.1, digitally signs them, submits to a real-time clearance service, and handles the entire lifecycle — retries with exponential backoff, webhook notifications, QR-stamped PDF generation, and a full audit trail. The frontend is a React/TypeScript dashboard giving finance teams live compliance visibility.

Containerized with Docker, tested with xUnit/Testcontainers/Playwright, deployed to Azure with PostgreSQL and CI/CD via GitHub Actions.

Live demo + full source in the comments."

**Suggested launch post (shorter, more casual tone for the feed):** "Jordan made e-invoicing mandatory for every business in 2025 — and most small businesses have no idea how to comply. So I built ClearVoice: an API that handles the whole clearance process automatically — signing, submission, retries, QR codes, the works. Docker + Azure + full test suite. Would love feedback from anyone who's touched fintech integrations. Demo link below 👇"

# 27. CV Presentation

**One-line project description:** Built a fintech compliance middleware (ASP.NET Core, PostgreSQL, Docker) that automates Jordan's mandatory e-invoicing clearance process end-to-end.

**CV bullets:** - Designed and built a Clean-Architecture ASP.NET Core Web API automating Jordan's mandatory e-invoicing clearance (UBL 2.1 mapping, digital signing, real-time submission), reducing a multi-week integration to a single API call. - Implemented a transactional-outbox retry pipeline with exponential backoff and idempotent, UUID-tracked resubmission, achieving reliable delivery without a dedicated message broker. - Built a webhook-based event system (HMAC-signed payloads) and a React/TypeScript dashboard giving finance teams real-time compliance visibility across cleared, failed, and pending invoices. - Enforced multi-tenant data isolation, Argon2id password hashing, rotating refresh tokens, and an immutable audit log; containerized with Docker Compose and deployed to Azure with PostgreSQL and GitHub Actions CI/CD.

**Skills demonstrated:** C#, ASP.NET Core, Clean Architecture, REST API design, PostgreSQL, Entity Framework Core, Docker, CI/CD (GitHub Actions), React, TypeScript, webhook/event-driven design, background workers, authentication/authorization (JWT), security engineering (OWASP), cloud deployment (Azure).

**Technologies:** C# / .NET 8, ASP.NET Core, PostgreSQL, Entity Framework Core, MediatR, React, TypeScript, TanStack Query, Docker, Docker Compose, GitHub Actions, Azure App Service, Azure Blob Storage, xUnit, Testcontainers, Playwright.

---

# 28. Interview Preparation

**Architecture questions:** - Why Clean Architecture over a simpler layered/MVC approach for this domain? (Answer around: business rules — invoice immutability, clearance state transitions — are complex enough to warrant isolating them from framework concerns; makes the domain testable without a database.) - Why the outbox pattern instead of directly calling the clearance service inline during the HTTP request? (Answer: reliability — the request should return fast and not fail the whole invoice submission if the external service is briefly slow; also guarantees at-least-once delivery even across process crashes.) - How would this scale from one server to ten? (Stateless API + `SKIP LOCKED` worker pattern allows horizontal scaling; database becomes the eventual bottleneck, addressed with read replicas/broker migration.)

**Database questions:** - Why `numeric` and not `float` for money fields? (Floating-point rounding errors are unacceptable in financial data; `numeric(14,3)` gives exact decimal representation.) - Why is `buyer_name`/`buyer_tax_id` nullable? (Per JoFotara rules, buyer details aren't required below a JOD 10,000 threshold — the schema reflects a real regulatory nuance, not an oversight.) - Why store `seller_snapshot` as denormalized JSON rather than a foreign key to a mutable `sellers` table? (An invoice must reflect the seller's details *at the time of issue*, immutably — if the seller's address changes next month, historical invoices must not silently change too.)

**API questions:** - Why 202 Accepted rather than 200/201 on invoice submission? (Clearance is asynchronous — the invoice is accepted for processing, not yet guaranteed cleared, at the moment the endpoint returns.) - How does the idempotency-key mechanism prevent duplicate invoices if a client's network call times out and they retry? (The same key returns the cached original response instead of creating a second invoice.)

**Security questions:** - Why Argon2id over bcrypt? (Memory-hard, more resistant to GPU/ASIC cracking, current OWASP recommendation.) - How is tenant data isolation actually enforced, not just assumed? (Global EF Core query filters scoped to the authenticated tenant ID, applied at the ORM level so no individual query can accidentally omit the filter.) - What happens if a webhook signing secret leaks? (It can be rotated from the dashboard; old signature verification immediately invalidates.)

**Design decisions to understand deeply:** the fresh-UUID-per-retry-attempt decision and why it matters for idempotency with the *external* clearance service specifically (not just internal idempotency); the transient-vs-validation failure classification and why they get different retry treatment; why the optional AI feature is display-only and never touches invoice data.

**Trade-offs to be ready to explain:** outbox table (simple, DB-coupled) vs. a real message broker (more infrastructure, better at scale) — and why starting with the simpler option was the right MVP call; synchronous UBL validation vs. fully async — why basic schema validation happens synchronously (fast user feedback) while the actual clearance call is async (external dependency shouldn't block the request thread).

---

# 29. Development Roadmap

**Phase 1 — Project Setup** - Tasks: initialize solution structure (Clean Architecture projects), set up Docker Compose skeleton, configure EF Core with PostgreSQL, set up GitHub repo + branch protection + CI skeleton. - Dependencies: none. - Definition of done: `docker compose up` boots an empty API returning `200` on `/health`.

**Phase 2 — Database** - Tasks: implement entities (§11), write initial EF Core migration, seed script for demo data. - Dependencies: Phase 1. - Definition of done: all tables created via migration; seed script populates a demo tenant with realistic invoices.

**Phase 3 — Authentication** - Tasks: register/login/refresh/logout endpoints, Argon2id hashing, JWT issuance, role-based policies, email verification flow. - Dependencies: Phase 2. - Definition of done: a user can register, verify email, log in, and receive a valid JWT; tenant isolation verified by an integration test.

**Phase 4 — Core Backend (Invoice Submission + Mock Clearance)** - Tasks: invoice submission endpoint, UBL 2.1 mapping service, digital signing service, mock clearance client, invoice status transitions. - Dependencies: Phase 3. - Definition of done: a submitted invoice is mapped, "signed," sent to the mock service, and returns a `CLEARED` or `FAILED` status correctly.

**Phase 5 — Frontend (Dashboard, Invoices, Auth screens)** - Tasks: build React app shell, auth screens, dashboard, invoices list/detail, onboarding wizard. - Dependencies: Phase 4 (needs a working API to integrate against). - Definition of done: a user can log in through the UI and see real invoice data from the API.

**Phase 6 — Integrations (Webhooks, API Keys)** - Tasks: webhook registration/dispatch, API key generation/auth, webhook signing. - Dependencies: Phase 4. - Definition of done: registering a webhook and submitting an invoice results in a correctly signed HTTP call to the test endpoint.

**Phase 7 — Automation (Retry Worker, Outbox, Scheduler)** - Tasks: implement outbox table + poller, retry backoff logic, daily digest job, nightly reconciliation job. - Dependencies: Phase 4, 6. - Definition of done: a deliberately-failing mock clearance call is automatically retried per the documented backoff schedule and eventually reaches `NEEDS_ATTENTION`.

**Phase 8 — Testing** - Tasks: write unit tests (domain logic), integration tests (Testcontainers), Playwright e2e tests for the two critical journeys. - Dependencies: Phases 3–7. - Definition of done: CI runs the full suite green; tenant-isolation and idempotency tests specifically pass.

**Phase 9 — Docker** - Tasks: finalize multi-stage Dockerfiles, polish `docker-compose.yml` for one-command local setup including seed data. - Dependencies: all prior phases functionally complete. - Definition of done: a fresh clone + `docker compose up` gives a fully working, seeded local instance with no manual steps.

**Phase 10 — CI/CD** - Tasks: GitHub Actions `ci.yml` (test/lint/scan on PR), `deploy.yml` (build/push/deploy to staging on merge to main). - Dependencies: Phase 9. - Definition of done: a merge to `main` automatically results in an updated staging deployment.

**Phase 11 — Cloud Deployment** - Tasks: provision Azure resources (App Service, PostgreSQL Flexible Server, Blob Storage, Key Vault, Static Web App), wire secrets, point CI/CD at them. - Dependencies: Phase 10. - Definition of done: the public demo URL is live and matches the local Docker experience.

**Phase 12 — Polish / Demo** - Tasks: seed a rich, realistic demo dataset; record the 2-minute walkthrough video; finalize README and architecture diagram; add the "Try Demo Account" one-click login button. - Dependencies: Phase 11. - Definition of done: the demo journey in §25 works exactly as scripted, start to finish, for a first-time visitor.

---

# 30. Final Product Blueprint

```
PRODUCT: ClearVoice — an e-invoicing compliance middleware/API and dashboard
         that automates Jordan's mandatory e-invoice clearance process.

PROBLEM: Since 1 April 2025, every invoice in Jordan must be validated
         (signed + QR-coded) by the national clearance system before it is
         legally valid; non-compliant invoices are not tax-deductible and
         carry penalties up to 500 JOD. Most SMEs cannot build this
         integration themselves.

USERS:   Business owners/finance admins, developers/integrators (POS/ERP
         vendors), tenant administrators, and (internally) a system
         operator role.

CORE WORKFLOW: Client submits invoice via REST API → ClearVoice validates,
         maps to UBL 2.1, digitally signs, submits to the clearance
         service → on success: QR-stamped PDF stored, webhook fired,
         dashboard updated → on failure: exponential-backoff retry with a
         fresh UUID per attempt, escalating to a human-facing
         "Needs Attention" queue after 5 attempts.

FRONTEND: React 18 + TypeScript, shadcn/ui + Tailwind, TanStack Query,
          React Hook Form + Zod, React Router.

BACKEND:  C# / ASP.NET Core 8, Clean Architecture + MediatR (CQRS-lite),
          EF Core, outbox-pattern background workers, Quartz.NET scheduler.

DATABASE: PostgreSQL — tenants, users, api_keys, invoices, invoice_lines,
          clearance_attempts, outbox_messages, webhook_endpoints,
          webhook_deliveries, audit_logs.

AI:       Optional, non-core only — an LLM "explain this rejection in
          plain language" feature, display-only, never modifies invoice
          data, always requires human review before resubmission.

INTEGRATIONS: National clearance service (mocked, feature-flagged real
          endpoint), transactional email, object storage, tenant
          webhooks, optional LLM provider.

SECURITY: Argon2id password hashing, rotating refresh tokens, JWT
          bearer auth, policy-based RBAC, tenant-scoped query filters,
          rate limiting, HMAC-signed webhooks, immutable audit log,
          secrets in managed vault, TLS everywhere.

DEVOPS:   Trunk-based Git flow, GitHub Actions CI/CD, multi-stage Docker
          builds, Docker Compose for one-command local dev, staged
          deployment with SHA-tagged rollback.

CLOUD:    Azure — App Service/Container Apps, PostgreSQL Flexible Server,
          Blob Storage, Key Vault, Static Web Apps, Application Insights.

MVP:      Invoice submission API, UBL 2.1 mapping, mock clearance,
          retry worker, QR-stamped PDF, dashboard, auth/RBAC, audit log.

FINAL VERSION: Multi-branch reporting, real clearance-service
          integration, white-labeling for POS vendors, SSO, HSM-backed
          signing, ERP connectors, Arabic-localized dashboard.
```